

Data Structure Lab 1 : Trees and Binary Trees

2024-2025

Topics

1. Create Position Interface

```
public interface Position<E> {  
    E element(); // Returns the element stored at this position  
}
```

2. Create Tree Interface

```
public interface Tree<E> {  
    Position<E> root();  
    int numChildren(Position<E> p);  
    Iterable<Position<E>> children(Position<E> p);  
    boolean isEmpty();  
    int size();  
    Iterator<E> iterator();  
    Iterable<Position<E>> positions();  
}
```

3. Create AbstractTree Class.

```
import java.util.Iterator;  
  
public abstract class AbstractTree<E> implements Tree<E> {  
    public boolean isEmpty() {  
        return size() == 0;  
    }  
  
    public int numChildren(Position<E> p) {  
        int count = 0;  
        for (Position<E> c : children(p)) {  
            count++;  
        }  
        return count;  
    }  
}
```

Data Structure Lab 1 : Trees and Binary Trees

2024-2025

```
public boolean isLeaf(Position<E> p) {  
    return numChildren(p) == 0;  
}
```

```
public int depth(Position<E> p) {  
    if (p == root()) return 0;  
    return 1 + depth(parent(p));  
}
```

```
public void printSubtree(Position<E> p, int d) {  
    for (int i = 0; i < d; i++) System.out.print(" ");  
    System.out.println(p.element());  
    for (Position<E> c : children(p)) {  
        printSubtree(c, d + 1);  
    }  
}
```

```
public void printTree() {  
    printSubtree(root(), 0);  
}
```

```
public abstract Position<E> parent(Position<E> p);  
}
```

4. Create BinaryTree Interface

```
public interface BinaryTree<E> extends Tree<E> {  
    Position<E> left(Position<E> p);  
    Position<E> right(Position<E> p);  
    Position<E> sibling(Position<E> p);  
}
```

5. Create AbstractBinaryTree Class

```
import java.util.ArrayList;
```

Data Structure Lab 1 : Trees and Binary Trees

2024-2025

```
import java.util.List;

public class LinkedBinaryTree<E> extends AbstractTree<E> implements
BinaryTree<E> {
    private static class Node<E> implements Position<E> {
        E element;
        Node<E> parent;
        Node<E> left;
        Node<E> right;

        public Node(E e, Node<E> p) {
            element = e;
            parent = p;
        }

        public E element() {
            return element;
        }
    }

    private Node<E> root;
    private int size = 0;

    public LinkedBinaryTree() {
        root = null;
    }

    public Position<E> root() {
        return root;
    }

    public int size() {
```

Data Structure Lab 1 : Trees and Binary Trees

2024-2025

```
        return size;
    }

    public Position<E> addRoot(E e) {
        if (!isEmpty()) throw new IllegalStateException("Tree already has a
root");
        root = new Node<>(e, null);
        size++;
        return root;
    }

    public Position<E> addLeft(Position<E> p, E e) {
        Node<E> node = validate(p);
        if (node.left != null) throw new IllegalStateException("Left child exists");
        node.left = new Node<>(e, node);
        size++;
        return node.left;
    }

    public Position<E> addRight(Position<E> p, E e) {
        Node<E> node = validate(p);
        if (node.right != null) throw new IllegalStateException("Right child
exists");
        node.right = new Node<>(e, node);
        size++;
        return node.right;
    }

    public E set(Position<E> p, E e) {
        Node<E> node = validate(p);
        E old = node.element;
        node.element = e;
    }
```

Data Structure Lab 1 : Trees and Binary Trees

2024-2025

```
        return old;
    }

    public E remove(Position<E> p) {
        Node<E> node = validate(p);
        if (numChildren(p) == 2) throw new IllegalArgumentException("Node
has two children");
        Node<E> child = (node.left != null) ? node.left : node.right;
        if (child != null) {
            child.parent = node.parent;
        }
        if (node == root) {
            root = child;
        } else {
            Node<E> parent = node.parent;
            if (node == parent.left) {
                parent.left = child;
            } else {
                parent.right = child;
            }
        }
        size--;
        return node.element;
    }

    private Node<E> validate(Position<E> p) {
        if (!(p instanceof Node)) throw new IllegalArgumentException("Invalid
position");
        return (Node<E>) p;
    }

    @Override
```

Data Structure Lab 1 : Trees and Binary Trees

2024-2025

```
public Position<E> left(Position<E> p) {  
    Node<E> node = validate(p);  
    return node.left;  
}
```

```
@Override  
public Position<E> right(Position<E> p) {  
    Node<E> node = validate(p);  
    return node.right;  
}
```

```
@Override  
public Position<E> sibling(Position<E> p) {  
    Node<E> node = validate(p);  
    Node<E> parent = node.parent;  
    if (parent == null) return null;  
    return (node == parent.left) ? parent.right : parent.left;  
}  
}
```

6. Create Linked Binary Tree Using Linked List structure (Nodes)

```
public class ArrayBinaryTree<E> {  
    private E[] tree;  
    private int size;  
    private static final int CAPACITY = 100;  
  
    public ArrayBinaryTree() {  
        tree = (E[]) new Object[CAPACITY];  
        size = 0;  
    }  
  
    public void add(E e) {  
        if (size == tree.length) throw new IllegalStateException("Tree is full");  
    }  
}
```

Data Structure Lab 1 : Trees and Binary Trees

2024-2025

```
        tree[size++] = e;
    }

    public E get(int index) {
        if (index < 0 || index >= size) throw new
IndexOutOfBoundsException();
        return tree[index];
    }

    public int leftChildIndex(int index) {
        return 2 * index + 1;
    }

    public int rightChildIndex(int index) {
        return 2 * index + 2;
    }

    public int parentIndex(int index) {
        return (index - 1) / 2;
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }
}
```

7. Implement Basic Methods of LinkedBinaryTree

- addRoot(E e)
- addLeft(Position<E> p ,E e)

Data Structure Lab 1 : Trees and Binary Trees

2024-2025

- addRight(Position<E> p ,E e)
- set(Position<E> p ,E e)
- remove(Position<E> p)

Homework

1. Implement the Array Based Binary Tree Data structure as it is described in chapter 8.

```
public class ArrayBinaryTree<E> {
    private E[] tree;
    private int capacity;
    private int size;

    @SuppressWarnings("unchecked")
    public ArrayBinaryTree(int capacity) {
        this.capacity = capacity;
        this.tree = (E[]) new Object[capacity];
        this.size = 0;
    }

    public void add(E value) {
        if (size >= capacity) {
            throw new IllegalStateException("Tree is full");
        }
        tree[size++] = value;
    }

    public E get(int index) {
        if (index < 0 || index >= size) {
            throw new IndexOutOfBoundsException("Index: " + index + ", Size: " + size);
        }
    }
}
```


Data Structure Lab 1 : Trees and Binary Trees

2024-2025

```
        }
        return tree[index];
    }

    public int getLeftChildIndex(int index) {
        return 2 * index + 1;
    }

    public int getRightChildIndex(int index) {
        return 2 * index + 2;
    }

    public int getParentIndex(int index) {
        return (index - 1) / 2;
    }

    public void printTree() {
        for (int i = 0; i < size; i++) {
            System.out.print(tree[i] + " ");
        }
        System.out.println();
    }

    public int size() {
        return size;
    }
}
```